Optimizing Cost and Performance for ML and AI Infrastructure

Slide 01: Title

Welcome to Domain 4, Part 2 of the theory-only learning series for the AWS Machine Learning Engineer Associate exam. In Part 1, we learned how to monitor models in production. In this video we focus on making those systems fast and affordable. We will cover choosing instance families, finding bottlenecks, building dashboards, planning capacity, controlling spend, and managing the cost of foundation models, tokens, and RAG systems. Let's jump in.

Slide 02: Choose the right instance family

The instance family is one of the biggest drivers of both cost and speed. CPU instances, like the ml dot c and ml dot m families, are cheap and work well for classical models like XGBoost or linear models. GPU instances are for heavy deep learning. The g5 and g6 families are popular for inference, while p4 and p5 are for very large training and inference jobs. AWS Inferentia is a chip designed by AWS specifically for inference. The inf2 instances offer a low cost per inference for deep learning and generative models, but you compile the model with the AWS Neuron SDK. AWS Graviton processors are Arm-based CPUs that give better price-performance for CPU inference, as long as your framework supports Arm. And when you are not sure, SageMaker Inference Recommender runs load tests on different instance types and recommends the best one for your latency and cost targets.

Slide 03: Find the bottleneck before you scale

Before adding capacity, find the real bottleneck. Amazon CloudWatch shows endpoint and instance metrics. High GPU utilization means the model is compute-bound. High memory utilization means the model may not fit well. And if model latency is low but overall latency is high, the delay is outside the model. That is where AWS X-Ray helps. X-Ray traces a single request across services, for example from Amazon API Gateway to AWS Lambda to the SageMaker AI endpoint, and shows exactly which hop is slow. Maybe it is a Lambda cold start or a slow database call, not the model at all. For agents, AgentCore Observability breaks each request into timed steps, so you can see whether a slow tool, a knowledge base lookup, or repeated retries is causing the delay. Exam tip: when a question asks which component is slow across services, think X-Ray.

Slide 04: Dashboards serve two different audiences

Dashboards turn metrics into decisions, but different people need different dashboards. On the left are Amazon CloudWatch dashboards. They give a near real-time operational view: latency, errors, utilization, and the status of alarms, all in one place. They are built for the engineers

who keep the system running. On the right is Amazon QuickSight, a business intelligence service. It is better for trends over weeks or months, such as cost per model, predictions per day, or business outcomes. A common pattern is to query AWS Cost and Usage Reports with Amazon Athena and visualize the results in QuickSight. Those dashboards are interactive and easy to share with managers and stakeholders. Exam tip: operational health in near real time points to CloudWatch, while business reporting and sharing with non-technical users points to QuickSight.

Slide 05: Plan capacity for latency, reliability, and cost

Capacity planning balances three goals: latency, reliability, and cost. Start by right-sizing. Choose the smallest instance that meets your latency target, using SageMaker Inference Recommender and load tests with realistic payloads. Review this regularly, because traffic and models change. Next, scale smart. Auto scaling should follow real demand. Set sensible minimum and maximum instance counts and tune the cooldown periods so the endpoint does not scale up and down too quickly. Where traffic allows, options like asynchronous inference and inference components can scale down to zero when idle. Finally, build resilience. For production endpoints, run at least two instances, because SageMaker AI spreads instances across Availability Zones. And check your service quotas, so you can actually scale up at peak times. A cheap endpoint that goes down at peak is not really cheap.

Slide 06: Control and reduce ML spend

Cost control has three parts. First, visibility. AWS Cost Explorer shows spending by service and by tag, and it can forecast future spend. Cost allocation tags are essential here. Tag every training job, endpoint, and notebook with the team or project, so you know exactly who is spending what. Second, limits. AWS Budgets sends alerts when costs approach a threshold, and budget actions can automatically apply restrictions, such as an IAM policy that stops new resources from being created. Service quotas also cap how much can be used. Third, purchase options. SageMaker AI Savings Plans give lower prices in exchange for a one or three year commitment to consistent usage. Managed Spot Training uses spare capacity at a large discount for interruptible training jobs. And for Bedrock, Provisioned Throughput suits steady, high-volume usage. Exam tip: steady usage points to Savings Plans, interruptible training points to Spot.

Slide 07: Foundation model costs are driven by tokens

With foundation models, cost is mostly about tokens. Input and output tokens are priced separately, and output tokens usually cost more. So keep prompts short, remove unnecessary instructions and examples, and set a maximum number of output tokens. You can track token counts per model in CloudWatch. Next, use the cost levers Amazon Bedrock offers. Prompt caching reuses repeated context, like a long system prompt, which reduces cost and latency. Batch inference processes many prompts offline at a lower price for supported models. And a

smaller model, or a distilled one, is often good enough for simple tasks. Intelligent prompt routing can send easy requests to a cheaper model automatically. Finally, remember the hidden costs. RAG systems pay for embedding tokens when documents are ingested and for vector store storage. Agents may call the model several times per request, so each extra step adds cost.

Slide 08: A cost and performance checklist

Here is the checklist for Part 2. One: pick CPU, GPU, Inferentia, or Graviton based on the workload, and let Inference Recommender help. Two: use CloudWatch and X-Ray to find the real bottleneck before adding capacity. Three: build CloudWatch dashboards for operators and QuickSight dashboards for business reporting. Four: right-size instances, auto scale with sensible limits, and spread production endpoints across Availability Zones. Five: tag resources, set budgets and alerts, and use Savings Plans or Spot where they fit. Six: manage token usage, embedding costs, and vector storage for generative AI workloads. In Part 3, we will finish Domain 4 by securing ML and AI workloads. Thanks for watching, and see you in the next one.